CMSC477: Robotics Perception and Planning
Fine Tuning YOLO for Custom Object Detection

Dr. Troi Williams, Khuzema Habib, Anukriti Singh

Quickstart

For your reference, we provide a complete example of a labeled dataset, trained the model, and tested on video the model was not trained on here. For outputs, check the folders under "runs/detect". You should still read the notes below before beginning on your own.

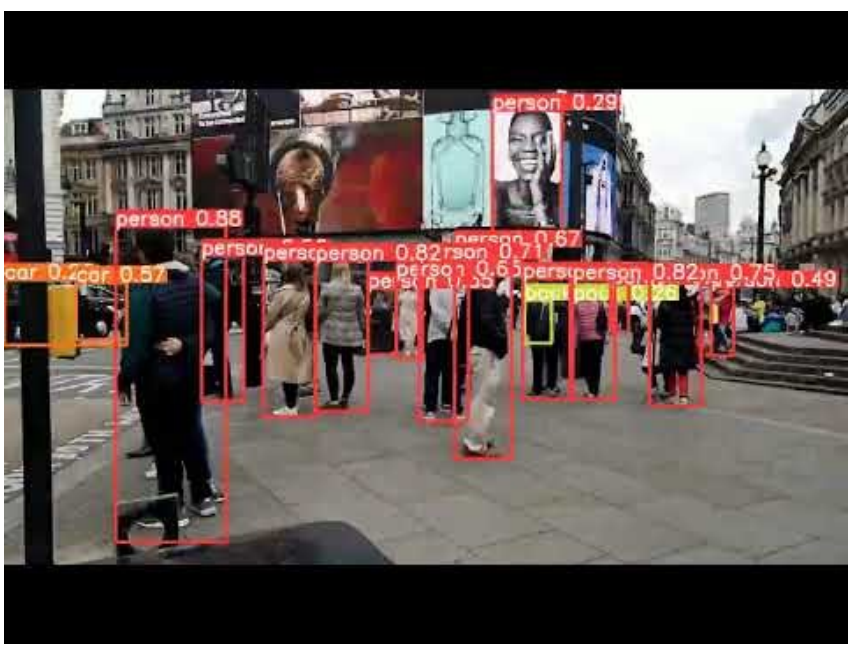
The full documentation for the ultralytic's YOLO package (which was used to make this tutorial) can be found here.

Introduction

YOLO (You Only Look Once) is a neural network that detects objects and provides the bounding box around them. It is widely used because of its speed and accuracy.

The model must be trained on examples of the objects to be detected. So, to detect Lego's and or RoboMaster's as needed for Projects 2 and 3 you will need to collect a small dataset (probably 100 images), label it, and fine-tune the model (train it).

Below is a video showing the model's output on a street scene:



Step 1: Create a Labeled Dataset

Refer to the sample dataset for a quick overview. Detailed instructions are provided below:
Note: We encourage teams to share their datasets with each other.

General Instructions:

- Capture a sequence of about 100 images of the object to be detected
  - Use the RoboMaster's camera
    - If you use your cell phone to capture the dataset, the model may still work, but it is best to use the same camera for training and testing
  - Be sure to vary the pose, lighting conditions, and background as possible
  - Make sure there are several images without any objects in them
    - Otherwise, the algorithm will learn to always detect an object
- Split the images into train and validation sets
  - Put about **80%** into a folder named "train"
  - Put the remaining in a folder named "validation"
  - Create a file called "names.txt". Put the name of each object on a separate row.
- Label the dataset
  - There are many tools available to create a labeled dataset for YOLO. We currently recommend Yolo Label.
    - Download and install Yolo Label following these instructions
  - Label the dataset by drawing bounding boxes
    - Refer to the Yolo Label README for detailed usage instructions and keyboard shortcuts
  - Images without any objects to be detected in them need to be labeled as well
    - Just viewing them in Yolo Label and then skipping to the next image will create this "empty" label.
- Create the dataset folder structure
  - Create a folder named "cmsc477_yolo"
    - Inside, create a folder named "datasets"
      - Put the "train" and "validation" folders in "datasets"
- Create a "data.yaml" file that configures the "datasets", "train", and "validation" directory and lists the objects in the dataset:
  - For example here is the "data.yaml" for our sample dataset:.

```yaml
path: ./ # After first run, YOLO expects all datasets to be in path+"/datsets"
train: robomaster_train
val: robomaster_validation
nc: 1  # Number of classes (1 for only "object")
names: ["robomaster"]
```

Step 2: Fine-Tune (train) the Model

From the “cmsc477_yolo” folder run:
“yolo detect train data=data.yaml model=v8s.pt imgsz=640 epochs=50”

This command will create a folder called “runs/detect/train{N}” where N is the number of times the command has been run. Keep track of this so that the testing below is run on the correct model parameters.

The flag “model=yolov8s.pt” means a pre-trained YOLOv8s model will be used as a starting point. The training procedure fine-tunes that model for our specific objects. This allows training to take much less time, and accuracy improves because the network was pre-trained on a diverse set of objects.

You can expect something like this:

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       1/50      1.98G      1.142      2.836      1.495         36        640: 100%|██████████| 1/1 [00:02<00:00,  2.05s/it]
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 1/1 [00:00<00:00,  3.43it/s]
                   all          7          7    0.00333          1     0.0556     0.0217

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       2/50      2.03G      1.115      3.031      1.481         31        640: 100%|██████████| 1/1 [00:00<00:00,  3.75it/s]
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 1/1 [00:00<00:00,  9.80it/s]
                   all          7          7    0.00333          1     0.0544     0.0167
```

Training Time: Training the sample dataset on WHO’s laptop took X seconds when using the NVIDIA GPU (CUDA) and X minutes with the cpu (add “device=cpu” to flags). If your team does not have a powerful enough computer to run the training in a reasonable amount of time, let Khuzema or Anukriti know and we will look into using Google Colab (which provides GPUs for free).

Validation: You can visualize the progress of YOLOs training by checking the corresponding “runs/detect/train{N}” folder. In particular, the result on the validation set are visualized there. Here is an example: *val_batch0_pred.jpg*

Model Selection: In the TAs testing, YOLOv8s appears to be more accurate than YOLOv11s for purposes of this class. Additionally, the small model (v8s) appears to be substantially more accurate than the nano model (v8n) because the v8n model has very low confidence when detecting the robomaster and sometimes misses it.

Bug Workaround: Yolo memorizes the location of the "datasets" folder and expects it to remain in the same place forever. If you need to reset this, run "yolo settings" and delete the settings file it prints the location of. Bug report is here.

Programmatic Training: It is possible to write a Python program that does the training, see here. This may be helpful if you train on Google Colab (where a GPU is available for free).

Step 3: Test the Model

To run live on the robomaster:
Use the code provided in Project 2 Description's Appendix. Just modify the model path to point to your new weights.

To run on a recorded video:
From the "cmsc477_yolo" folder run:
yolo detect predict model=runs/detect/train{N_train}/weights/best.pt source=test_video.mp4
The results will be in "runs/detect/predict{N_predict}"
In the provided sample, the video is in "runs/detect/predict/test_video.avi"
Performance: GPU: 5-6ms per frame, CPU: 50-60ms per frame

To run a folder of images:
From the "cmsc477_yolo" folder run:
yolo detect predict model=runs/detect/train{N_train}/weights/best.pt source=datasets/robomaster_validation
The results will be in "runs/detect/predict{N_predict}"